



Universidad de las Fuerzas Armadas ESPE

Modelos Discretos para la Ingeniería

NRC: 30623

CARRERA: Ingeniería en Tecnologías de la Información

INTEGRANTES:

- ✓ Ayala Maldonado Eatan Joao
- ✓ Cuichan Males Lenin Esteban
- ✓ Loachamin Fuel Matias Valentino
- ✓ Palacios Campoverde Martin Sebastián

FECHA: Martes, 23 de Junio de 2026

PERÍODO: SI-ABR26-AGO26

SISTEMA LÓGICO DE ASIGNACIÓN DE RECURSOS EN UN CENTRO DE DATOS

SANGOLQUÍ – ECUADOR



Contenidos

1.	Introducción	4
2.	Objetivos	5
2.1.	Objetivo General.....	5
2.2.	Objetivos Específicos	5
3.	Arquitectura del Código	6
3.1.	Definición de Tipos de Datos Abstractos (TDAs).....	6
3.2.	Estructura de Cola Enlazada	6
4.	Análisis Teórico y Modelos Discretos	8
4.1.	Teoría de Conteo y Espacio Muestral.....	8
4.2.	Teoría de Conjuntos y Probabilidad Discreta.....	11
4.2.1.	<i>Definición Espacio Muestral</i>	11
4.2.2.	<i>Espacio muestral:</i>	12
4.2.3.	<i>Clasificación de combinaciones</i>	12
4.2.4.	<i>Representación mediante conjuntos:</i>	13
4.2.5.	<i>Calculo Probabilidad Discreta</i>	13
4.2.6.	<i>Verificación mediante Kolmogorov</i>	14
4.3.	Relaciones de Orden Parcial y Diagrama de Hasse.....	14
5.	Análisis de Complejidad Algorítmica	20
5.1.	Complejidad Temporal	20
5.2.	Complejidad Espacial.....	21
5.3.	Relación con los Modelos Discretos (Diagrama de Hasse)	22
6.	Pruebas y Resultados.....	23
7.	Conclusiones	25
8.	Recomendaciones	26
9.	Bibliografía.....	26



Tabla de contenidos - Tablas

Tabla 1 - Definición Espacio Muestral	11
Tabla 2 - Abreviaturas	11
Tabla 3 - Clasificación de combinaciones	12
Tabla 4 - Clasificación de eventos	12
Tabla 5 - Porcentajes de Probabilidad.....	13

Tabla de contenidos - Ilustraciones

Ilustración 1 - Clasificación del espacio muestral de 12 combinaciones lógicas	16
Ilustración 2 - Solicitudes manuales.....	17
Ilustración 3 - Evaluación Lógica.....	17
Ilustración 4 - Probabilidad Secuencial	18
Ilustración 5 - Separación en vectores	18
Ilustración 6 - Probabilidad en conjuntos.....	19
Ilustración 7 - Bucle While (comprueba si la cola está vacía).....	21
Ilustración 8 - Estado del Servidor.....	23
Ilustración 9 - Ingreso de solicitudes.....	24
Ilustración 10 - Insetado de solicitud crítica	24
Ilustración 11 - Estado de solicitudes	25



1. Introducción

En la administración de infraestructuras tecnológicas modernas, la asignación eficiente de recursos críticos como CPU y memoria RAM constituye uno de los desafíos operacionales más complejos. Cuando múltiples peticiones convergen simultáneamente hacia un servidor con capacidades físicas limitadas, se vuelve indispensable implementar estructuras de datos lineales que optimicen el flujo de información en el heap del sistema mediante el uso estricto de memoria dinámica y punteros nativos (Joyanes Aguilar, Estructuras de datos en C++ (5a ed.), 2020). Bajo este enfoque, el diseño de un Tipo de Dato Abstracto (TDA) basado en una cola enlazada con política de servicio FIFO (First In, First Out) garantiza que la recepción de procesos no incurra en desbordamientos de memoria ni pérdidas de datos, previendo al software de una base arquitectónica robusta y de alta modularidad para la gestión de solicitudes en tiempo real.

Paralelamente, la toma de decisiones que gobierna el comportamiento del servidor no se ejecuta de forma arbitraria, sino que es estructurada a través de los principios fundamentales de la matemática discreta. Al modelar las condiciones de frontera de la infraestructura mediante la teoría de conjuntos, el principio del producto y los modelos de probabilidad clásica así es posible mapear con exactitud el espacio muestral del sistema para predecir probabilísticamente su rendimiento ante diferentes estados de carga (Rosen, 2012). Asimismo, la jerarquización y el nivel de prioridad de los veredictos emitidos (aprobación, espera o rechazo) son analizados formalmente bajo el enfoque de los Conjuntos Parcialmente Ordenados (Posets), empleando el Diagrama



de Hasse para demostrar analíticamente cómo las propiedades de reflexividad, antisimetría y transitividad configuran de manera inequívoca la lógica de control en los sistemas informáticos.

2. Objetivos

2.1. Objetivo General

Desarrollar un sistema informático modular de simulación para la asignación de recursos de hardware en un centro de datos en el lenguaje C++, mediante el uso de estructuras de datos lineales dinámicas y motores de decisión lógicos, con el fin de evaluar probabilística y jerárquicamente el comportamiento de las solicitudes de los usuarios basándose en modelos discretos.

2.2. Objetivos Específicos

- Implementar el Tipo de Dato Abstracto (TDA) Cola mediante clases y punteros nativos en C++, garantizando una gestión eficiente de la memoria dinámica bajo la política FIFO.
- Mapear el espacio muestral del sistema mediante el principio del producto y la teoría de conjuntos para calcular las probabilidades discretas de las doce combinaciones posibles.
- Modelar la jerarquía de decisiones del servidor (aprobación, espera y rechazo) como un conjunto parcialmente ordenado (Poset) mediante el Diagrama de Hasse.



3. Arquitectura del Código

3.1. Definición de Tipos de Datos Abstractos (TDAs)

Un Tipo de Dato Abstracto (TDA) constituye un modelo matemático compuesto por un conjunto de datos y las operaciones definidas sobre ellos. Su diseño se centra exclusivamente en definir qué hace el tipo de dato y no en cómo se implementa internamente, separando la interfaz lógica de su representación física en memoria mediante el principio de encapsulamiento (Dale & Walker, 1996). En el paradigma de la programación Orientada a Objetos, los TDAs se construyen mediante clases, aislando los atributos de forma privada para proteger la integridad de los datos en el heap y obligando a que cualquier interacción externa se realice estrictamente a través de métodos públicos (Stroustrup, 2013).

Para el desarrollo de este proyecto, el software se cimenta sobre la interacción de cuatro TDAs modulares implementados de forma nativa en C++. El TDA Solicitud encapsula la identidad y los límites físicos libres de la infraestructura; el TDA CentroDatos actúa como el motor de reglas encargado de evaluar las decisiones del entorno; y, finalmente, el TDA ColaEnlazada gestiona de forma dinámica el flujo y la secuencia temporal de las solicitudes utilizando punteros de memoria.

3.2. Estructura de Cola Enlazada

Las colas enlazadas son estructuras de datos dinámicas lineales especializadas que restringen el acceso a sus elementos bajo el principio formal FIFO (First In, First Out). A diferencia de las listas enlazadas genéricas – las cuales se administran desde un nodo cabeza y permiten inserciones o eliminaciones en posiciones arbitrarias –, una cola



dinámica garantiza que el primer elemento en ser almacenado en la memoria heap sea estrictamente el primero en ser procesado y removido de la secuencia (Joyanes Aguilar & Zahonero Martinez, Estructuras de datos en C++, 2008).

Los elementos se conectan mediante nodos independientes que no requieren contigüidad física en la memoria del sistema, ya que la continuidad lógica se establece a través de punteros nativos que enlazan cada nodo con la dirección del siguiente en el flujo de ejecución. En esta estructura, las operaciones fundamentales de manipulación de memoria se restringen a tres compartimientos lineales:

- Encolar (Insertar): Agregar nuevos nodos exclusivamente al final de la secuencia.
- Desencolar (Eliminar): Remover y retornar el nodo que se encuentra en el frente de la secuencia.
- Recorrer: Acceder secuencialmente a los nodos para verificar el estado de la estructura o liberar la memoria dinámica del sistema operativo mediante el operador delete.

En el proyecto, la clase ColaEnlazada implementa esta lógica para almacenar y gestionar de forma dinámica las solicitudes de recursos de CPU y RAM. Esto permite que el sistema mantenga un flujo de espera ordenado antes de que el motor evalúe analíticamente las peticiones y las asigne a los conjuntos de respuesta correspondientes.



4. Análisis Teórico y Modelos Discretos

4.1. Teoría de Conteo y Espacio Muestral

Los principios fundamentales de conteo son reglas básicas de la combinatoria que permiten calcular el número de formas en que pueden ocurrir ciertos eventos, sin necesidad de enumerar cada posibilidad una por una. Se utilizan en situaciones de conteo y probabilidad, y constituyen la base para técnicas más avanzadas como permutaciones y combinaciones (Castañón, Rodríguez, & Del Rosario Gámez Aguilar, 2026).

Según (Castañón, Rodríguez, & Del Rosario Gámez Aguilar, 2026) el principio aditivo, se refiere a que si un proceso puede realizarse de dos maneras mutuamente excluyentes (no pueden ocurrir al mismo tiempo), entonces el número total de maneras de realizarlo es la suma de ambas posibilidades.

En el principio de multiplicación, si un proceso se compone de dos pasos sucesivos y el primero puede hacerse de m formas y el segundo de n formas, entonces el número total de formas de realizar ambos pasos es la multiplicación $m \times n$ formas (Castañón, Rodríguez, & Del Rosario Gámez Aguilar, 2026).

Por último, el principio de inclusión-exclusión dice que cuando las opciones no son mutuamente excluyentes, ósea que pueden coincidir, se debe restar la intersección (Castañón, Rodríguez, & Del Rosario Gámez Aguilar, 2026).

Se define el espacio muestral y se denota por Ω , como el conjunto de todos los posibles resultados de un experimento aleatorio.



Para aplicarlo en el sistema de asignación de recursos del centro de datos, se usaron 3 variables para la toma de decisiones:

Estado de Carga del Servidor:

- Baja
- Media
- Alta

Disponibilidad de Recursos:

- Disponible
- No disponible

Tipo de Solicitud:

- Prioritaria
- Normal

Estas 3 variables a pesar de que las analizamos en conjunto para tomar las decisiones de asignación de recursos son independientes entre sí, por lo tanto, se usa el principio de multiplicación, para lograr determinar el número total de escenarios posibles.

$$n = 3 \times 2 \times 2 = 12$$

Este cálculo rápido, se interpreta, de que existe 12 combinaciones posibles durante la evaluación del sistema.

El espacio muestral de todas las posibles combinaciones es el siguiente:



$$\Omega = \{ (B, D, P), (B, D, N), (B, ND, P), (B, ND, N), (M, D, P), (M, D, N),$$

$$(M, ND, P), (M, ND, N), (A, D, P), (A, D, N), (A, ND, P), (A, ND, N) \}$$

Donde:

- B = Baja
- M = Media
- A = Alta
- D = Disponible
- ND = No disponible
- P = Prioritaria
- N = Normal

La cardinalidad del espacio muestral es:

$$|\Omega| = 12$$

Luego de definir e identificar las 12 combinaciones posibles en el sistema, este evalúa las condiciones lógicas para determinar el estado de la solicitud, entre aprobada, espera o rechazada, las condiciones que ocupa el sistema son las siguientes:

- P: El servidor está operativo.
- Q: El servicio es crítico.
- R: Existen recursos disponibles.

Las reglas de inferencia implementadas son:

- $P \wedge Q \wedge R \rightarrow \text{Aprobada}$

- $P \wedge \neg Q \wedge R \rightarrow \text{Espera}$
- $\neg P \vee \neg R \rightarrow \text{Rechazada}$

El orden en que se evalúan las proposiciones no modifica el resultado final, ya que, gracias a las propiedades de la lógica proposicional, se puede aplicar la propiedades conmutativa y asociativa, de forma que independientemente del orden en que se evalúen las condiciones, la decisión que tome el sistema será la misma.

4.2. Teoría de Conjuntos y Probabilidad Discreta

4.2.1. Definición Espacio Muestral

Tabla 1 - Definición Espacio Muestral

Variable	Opciones
Estado de carga	Baja, Media, Alta
Recursos	Disponible, No disponible
Tipo de solicitud	Prioritaria, Normal

Aplicando el principio multiplicativo:

$$|\Omega| = 3 \times 2 \times 2 = 12$$

El espacio muestral esta formado por 12 combinaciones posibles.

Se usan las abreviaturas:

Tabla 2 - Abreviaturas

Símbolo	Significado
B	Carga baja
M	Carga media
A	Carga alta
D	Recursos disponibles
ND	Recursos no disponibles



P	Solicitud prioritaria
N	Solicitud normal

4.2.2. Espacio muestral:

$$\Omega = \{ (B, D, P), (B, D, N), (B, ND, P), (B, ND, N), (M, D, P), (M, D, N), (M, ND, P), (M, ND, N), (A, D, P), (A, D, N), (A, ND, P), (A, ND, N) \}$$

4.2.3. Clasificación de combinaciones

Cada combinación se clasificará en uno de los tres estados que existen: espera, aprobado o rechazado.

Tabla 3 - Clasificación de combinaciones

Caso	Carga	Recursos	Tipo de solicitud	Decisión
1	Baja	Disponible	Prioritaria	Aprobada
2	Baja	Disponible	Normal	Aprobada
3	Baja	No disponible	Prioritaria	Espera
4	Baja	No disponible	Normal	Rechazada
5	Media	Disponible	Prioritaria	Aprobada
6	Media	Disponible	Normal	Espera
7	Media	No disponible	Prioritaria	Espera
8	Media	No disponible	Normal	Rechazada
9	Alta	Disponible	Prioritaria	Espera
10	Alta	Disponible	Normal	Espera
11	Alta	No disponible	Prioritaria	Espera
12	Alta	No disponible	Normal	Rechazada

Donde se clasificarán los eventos de la siguiente forma:

Tabla 4 - Clasificación de eventos

Evento	Casos favorables	Combinaciones
Ap = Aprobada	3	(B,D,P), (B,D,N), (M,D,P)

Es = Espera	6	(B,ND,P), (M,D,N), (M,ND,P), (A,D,P), (A,D,N), (A,ND,P)
Re = Rechazada	3	(B,ND,N), (M,ND,N), (A,ND,N)
Total	12	Ω

4.2.4. Representación mediante conjuntos:

$$Ap = \{ (B,D,P), (B,D,N), (M,D,P) \}$$

$$Es = \{ (B,ND,P), (M,D,N), (M,ND,P), (A,D,P), (A,D,N), (A,ND,P) \}$$

$$Re = \{ (B,ND,N), (M,ND,N), (A,ND,N) \}$$

Donde los tres subconjuntos cumplen con que son mutuamente excluyentes y que la combinación del espacio muestral pertenece a uno de los tres resultados.

$$Ap \cap Es = \emptyset, Ap \cap Re = \emptyset, Es \cap Re = \emptyset$$

$$Ap \cup Es \cup Re = \Omega$$

4.2.5. Calculo Probabilidad Discreta

Dada la fórmula:

$$P(E) = \frac{\text{casos favorables}}{\text{casos totales}}$$

Donde se obtiene la siguiente tabla:

Tabla 5 - Porcentajes de Probabilidad

Evento	Casos favorables	Probabilidad fraccionaria	Porcentaje
Aprobada	3	$3/12 = 1/4$	25%
Espera	6	$6/12 = 1/2$	50%
Rechazada	3	$3/12 = 1/4$	25%

$$P(\text{Aprobada}) = \frac{3}{12} = 0.25 = 25$$

$$P(\text{Espera}) = \frac{6}{12} = 0.50 = 50$$

$$P(\text{Rechazada}) = \frac{3}{12} = 0.25 = 25\%$$

4.2.6. Verificación mediante Kolmogorov

Dado que todas las probabilidades son mayores o iguales a cero, obtenemos que:

$$P(\text{Aprobada}) \geq 0, \quad P(\text{Espera}) \geq 0, \quad P(\text{Rechazada}) \geq 0$$

Dado que la probabilidad del espacio muestral debe ser igual a 1, y ya se calculó la probabilidad de cada uno tenemos:

$$P(\Omega) = 1$$

$$P(\text{Aprobada}) + P(\text{Espera}) + P(\text{Rechazada}) = 1$$

$$\frac{3}{12} + \frac{6}{12} + \frac{3}{12} = \frac{12}{12} = 1$$

$$25\% + 50\% + 25\% = 100\%$$

Y finalmente como se trata con eventos mutuamente excluyentes tenemos que la probabilidad de la unión de cada evento es igual a su suma individual:

$$P(\text{Ap} \cup \text{Es} \cup \text{Re}) = P(\text{Ap}) + P(\text{Es}) + P(\text{Re})$$

4.3. Relaciones de Orden Parcial y Diagrama de Hasse

El estudio de cómo se organizan y comparan los elementos de un conjunto existe en múltiples escenarios en la lógica, el álgebra y las ciencias de la computación, donde esta propiedad no se cumple obligatoriamente, dando origen a las relaciones de orden parcial.



En el campo de las matemáticas discretas, una relación binaria $\leq$ sobre un conjunto P se clasifica como una relación de orden parcial si cumple con las tres propiedades axiomáticas:

- ❖ *Reflexividad: todo elemento está relacionado consigo mismo ($a \leq a$)*
- ❖ *Antisimétrica: si $a \leq b$ y $b \leq a$ entonces $a = b$*
- ❖ *Transitividad: si $a \leq b$ y $b \leq c$, entonces $a \leq c$*

Cuando un conjunto está dotado en esta relación la estructura resultante se denomina conjunto parcialmente ordenado o poset, el termino parcial proviene de que pueden existir elementos dentro del conjunto que no son comparables entre si (Williams, 2024).

Para la representación gráfica de un conjunto parcialmente ordenado mediante grafos convencionales suele resultar redundante y visualmente complejo debido a la presencia de bucles reflexivos y aristas transitivas. Como solución (Williams, 2024) propone el Diagrama de Hasse que simplifica la visualización apoyándose en un concepto clave, la relación de cobertura, se dice que un elemento b cubre a un elemento a (denotado como $a < (\text{con un punto}) b$) si $a < b$ y no existe ningún elemento intermedio x tal que $a < x < b$. De modo que las relaciones de orden parcial se conectan directamente con el Diagrama de Hasse al eliminar toda la información redundante, se dibujan únicamente segmentos de línea recta entre los nodos que cumplen la relación cobertura, posicionando el elemento mayor en un nivel vertical más alto.

El principio multiplicativo de conteo demuestra la existencia de un espacio muestral cerrado 12 combinaciones estables. Para estructurar formalmente las decisiones tomadas por el sistema de asignación de recursos (Aprobar, Espera, Rechazar), se implementó un modelo basado en la teoría de Conjuntos Parcialmente Ordenados (Posets), se clasifica el universo de estados, identificando una distribución teórica de 3 resultados aprobados (25%), 6 resultados en espera (50%) y 3 resultados rechazados (25%).

```
Seleccione una opcion: 6

CLASIFICACION DE COMBINACIONES
1. Baja | Disponible | Prioritaria -> Aprobada
2. Baja | Disponible | Normal -> Aprobada
3. Baja | No Disponible | Prioritaria -> Espera
4. Baja | No Disponible | Normal -> Rechazada
5. Media | Disponible | Prioritaria -> Aprobada
6. Media | Disponible | Normal -> Espera
7. Media | No Disponible | Prioritaria -> Espera
8. Media | No Disponible | Normal -> Rechazada
9. Alta | Disponible | Prioritaria -> Espera
10. Alta | Disponible | Normal -> Espera
11. Alta | No Disponible | Prioritaria -> Espera
12. Alta | No Disponible | Normal -> Rechazada

RESUMEN DE LAS 12 COMBINACIONES
Aprobadas: 3
Espera: 6
Rechazadas: 3
```

Ilustración 1 - Clasificación del espacio muestral de 12 combinaciones lógicas

- ❖ Evidencia del Universo: Muestra la simulación automática de las 12 combinaciones posibles del sistema (cruzar carga, recursos y prioridad).
- ❖ Filtro Jerárquico: El programa aplica las reglas lógicas asignando cada caso a un estado del orden parcial: Aprobada, Espera o Rechazada.

- ❖ Validación Teórica: El resumen final arroja 3 Aprobadas (25%), 6 en Espera (50%) y 3 Rechazadas (25%). Esto demuestra computacionalmente que el código coincide al 100% con los cálculos probabilísticos del informe y justifica que el nivel intermedio (Espera)

```
Seleccione una opcion: 1
TIPO (Prioritaria/Normal): Normal
CPU requerida: 2
RAM requerida: 4
Servicio critico? (1=SI 0=NO): 0
Solicitud 1 agregada correctamente.
```

Ilustración 2 - Solicitudes manuales

```
Seleccione una opcion: 1
TIPO (Prioritaria/Normal): Prioritaria
CPU requerida: 4
RAM requerida: 8
Servicio critico? (1=SI 0=NO): 0
Solicitud 2 agregada correctamente.
```

sea el más denso en el Diagrama de Hasse.

Ingreso de datos, se muestra el registro manual de dos solicitudes en la cola del servidor con distintos parámetros de hardware (CPU, RAM y Prioridad).

```
--- PROPOSICIONES ---
P (Servidor operativo): V
Q (Servicio critico): F
R (Recursos disponibles): V
Solicitud 1 -> Espera

--- PROPOSICIONES ---
P (Servidor operativo): V
Q (Servicio critico): F
R (Recursos disponibles): V
Solicitud 2 -> Espera
```

Ilustración 3 - Evaluación Lógica

El sistema evalúa las proposiciones lógicas de control (P, Q, R) para cada proceso. Al contrastarlas con las reglas del negocio, ambas solicitudes resultan asignadas al estado de Espera.

```
Seleccione una opcion: 4  
  
PROBABILIDADES  
P(Aprobada) = 0%  
P(Espera) = 100%  
P(Rechazada) = 0%
```

Ilustración 4 - Probabilidad Secuencial

Refleja la estadística real de la simulación en caliente del Centro de Datos. Al estar los dos únicos procesos retenidos, el sistema calcula dinámicamente: $P(\text{Espera}) = 100\%$, mientras que $P(\text{Aprobada}) = 0\%$ y $P(\text{Rechazada}) = 0\%$. Esto demuestra que la probabilidad frecuencial varía según el comportamiento del usuario y difiere de la probabilidad teórica estática.

```
Seleccione una opcion: 9  
  
A = { }  
B = { 1 2 }  
C = { }
```

Ilustración 5 - Separación en vectores

El programa segrega los identificadores reales de los procesos. Al haber quedado ambos en espera, el vector B almacena los ID {1, 2}, mientras que los conjuntos A (Aprobadas) y C (Rechazadas) permanecen vacíos {}.

Aplicando la relación de cobertura, se suprimieron los bucles reflexivos y las aristas transitivas redundantes para construir el Diagrama de Hasse del sistema. La jerarquía se organiza verticalmente de abajo hacia arriba en tres niveles fijos: el conjunto C en la base (mínima prioridad), el conjunto B en el centro (absorbiendo la

mayor densidad probabilística con el 50%) y el conjunto A en la cúspide (máxima prioridad y asignación inmediata).



Ilustración 6 - Probabilidad en conjuntos

El Conjunto A debe estar físicamente arriba de B, y B arriba de C, se demuestra $C \leq B \leq A$ en la jerarquía de asignación de recursos del centro de datos.

Dado que la altura del diagrama de decisiones es constante y está limitada a un camino crítico de 3 niveles (A, B, C), la evaluación de una solicitud individual toma un tiempo constante $O(1)$. Por lo tanto, al procesar una cola de n solicitudes pendientes mediante estructuras enlazadas, el sistema ejecuta un comportamiento lineal tanto en



complejidad temporal como espacial de $O(n)$, demostrando ser una solución óptima y escalable para la administración del Centro de Datos.

El Software valida la Teoría Matemática: Al clasificar las 12 combinaciones posibles, el código confirma la mayor densidad en el estado de Espera (50%), lo que justifica geométricamente que la estructura en el centro del Diagrama de Hasse de ser ensanchada.

Diferencia entre Probabilidad Teórica y Frecuencial: La teoría dice que la probabilidad de quedar en espera es del 50%, el uso real con dos solicitudes específicas empujó el sistema al extremo (100% de Espera). Los vectores dinámicos reflejan con precisión matemática esta realidad al almacenar los procesos en el conjunto $B = \{1, 2\}$, dejando A y C vacíos.

5. Análisis de Complejidad Algorítmica

El análisis de la eficiencia del software desarrollado se evalúa bajo los criterios de complejidad temporal (costo en tiempo de ejecución) y complejidad espacial (consumo de memoria del sistema), utilizando la notación asintótica Big O. El comportamiento del sistema directamente ligado a la naturaleza dinámica de la estructura de datos elegida para el flujo de solicitudes.

5.1. Complejidad Temporal

El análisis del tiempo de ejecución se divide en las dos operaciones críticas del ciclo de vida del TDA Cola Enlazada dentro del sistema:

- Operación de Encolado (encolar): El ingreso de una nueva solicitud al sistema presenta una complejidad de $O(1)$ (tiempo constante). Al mantener un

puntero directo al último nodo de la secuencia (Rear o Back), la inserción de un nuevo elemento en la memoria heap y la reasignación del puntero de enlace no dependen del volumen de elementos previos en la cola.

- Operación de Desencolado y Evaluación (procesarSolicitudes): El procesamiento y vaciado completo de la estructura lineal presenta una complejidad de $O(N)$ en el peor de los casos, donde N representa el número total de solicitudes almacenadas.

```
while(!cola.estaVacia()){
    Solicitud* solicitud = cola.desencolar();
    string resultado = centro.evaluarSolicitud(solicitud);
    cout << "Solicitud "<< solicitud->getId()<< " -> "<< resultado<< endl;
    if(resultado=="Aprobada"){
        aprobadas++;
        A.push_back(solicitud->getId());
    }else if(resultado=="Espera"){
        espera++;
        B.push_back(solicitud->getId());
    }else{
        rechazadas++;
        C.push_back(solicitud->getId());
    }
    delete solicitud;
}
```

Ilustración 7 - Bucle While (comprueba si la cola está vacía)

El bucle *while* (*!cola.estaVacia()*) realiza exactamente N iteraciones. En cada ciclo, las operaciones de extracción del nodo, la evaluación lógica de fronteras físicas en la clase CentroDatos y la liberación de memoria mediante el operador delete se ejecutan en tiempo constante $O(1)$. Por lo tanto, el tiempo total escala de forma estrictamente lineal respecto al número de entradas:

$$T(N) = N \times O(1) = O(N)$$

5.2. Complejidad Espacial

La complejidad espacial evalúa el uso de memoria asignada dinámicamente durante el tiempo de ejecución del programa:

- Consumo Dinámico: El software opera con una complejidad espacial de $O(N)$ (espacio lineal). Al utilizar una estructura enlazada en lugar de un arreglo estático, la memoria no se reserva de forma contigua ni anticipada. Cada solicitud instanciada mediante el operador *new* genera un nodo aislado en el heap que consume memoria de manera directamente proporcional a la cantidad de peticiones concurrentes en el Centro de Datos.
- Optimización de Memoria: La implementación asegura la liberación inmediata de los recursos espaciales. Una vez que la solicitud es evaluada por el motor de reglas y clasificada en su respectivo veredicto, el nodo es destruido explícitamente (*delete solicitud*). Esto reduce la persistencia espacial a un máximo de $O(N)$ elementos en espera y garantiza un retorno a $O(1)$ al finalizar el vaciado de la cola, previniendo fugas de memoria (memory leaks).

5.3. Relación con los Modelos Discretos (Diagrama de Hasse)

El universo de las 12 combinaciones lógicas se particiona en tres conjuntos excluyentes según el veredicto del software. Definimos el conjunto $P = \{A, B, C\}$ bajo la relación de orden ($\leq$), definida como: “tiene menor o igual viabilidad de asignación inmediata de recursos”.

Propiedades del Poset

La estructura lineal del diagrama demuestra formalmente las tres propiedades de un conjunto parcialmente ordenado:

1. **Reflexividad:** Cada conjunto se compara consigo mismo ($A \leq A, B \leq B, C \leq C$).

2. **Antisimetría:** Como el conjunto en Espera tiene menor prioridad que el Aprobado ($B \leq A$), la inversa no puede ser cierta porque son conjuntos con veredictos distintos ($A \neq B$).
3. **Transitividad:** Si Rechazada tiene menor viabilidad que Espera ($C \leq B$), y Espera menor que Aprobada ($B \leq A$), se concluye que:

$$C \leq B \wedge B \leq A \rightarrow C \leq A$$

Análisis de Cotas y Elementos

- **Elemento Máximo (Cota Superior):** El **Conjunto A (Aprobada)**. Está en la cúspide porque representa el estado ideal de despacho inmediato.
- **Elemento Mínimo (Cota Inferior):** El **Conjunto C (Rechazada)**. Se ubica en la base al representar la denegación absoluta de recursos.

La arista que conecta a los bloques elimina las líneas redundantes por reflexividad y transitividad. Al ser una estructura puramente vertical ($C \leq B \leq A$), este diagrama de Hasse representa una Cadena de Orden Total, conectando el motor del código con la jerarquía matemática formal.

6. Pruebas y Resultados

```
--- ESTADO DEL SERVIDOR ---  
Carga: Media  
CPU Disponible: 16 Cores  
RAM Disponible: 32 GB  
Operativo: SI  
-----
```

Ilustración 8 - Estado del Servidor

```
=====
SISTEMA DE ASIGNACION DE RECURSOS
=====
1. Ingresar solicitud
2. Procesar solicitudes
3. Mostrar resumen
4. Mostrar probabilidades
5. Mostrar universo de combinaciones
6. Mostrar clasificacion de combinaciones
7. Mostrar estado del servidor
8. Mostrar solicitudes pendientes
9. Mostrar conjuntos A, B y C
0. Salir

Seleccione una opcion: 1

TIPO (Prioritaria/Normal): Normal
CPU requerida: 4
RAM requerida: 8
Servicio critico? (1=SI 0=NO): 0

Solicitud 1 agregada correctamente.
```

Ilustración 9 - Ingreso de solicitudes

En este escenario se demuestra el comportamiento del sistema bajo condiciones óptimas de hardware. Al encontrarse el servidor en carga baja y disponer de recursos suficientes, el TDA Cola procesa el nodo en tiempo $O(1)$ y el motor aprueba la solicitud de forma inmediata, descontando los recursos del servidor.

```
=====
SISTEMA DE ASIGNACION DE RECURSOS
=====
1. Ingresar solicitud
2. Procesar solicitudes
3. Mostrar resumen
4. Mostrar probabilidades
5. Mostrar universo de combinaciones
6. Mostrar clasificacion de combinaciones
7. Mostrar estado del servidor
8. Mostrar solicitudes pendientes
9. Mostrar conjuntos A, B y C
0. Salir

Seleccione una opcion: 1

TIPO (Prioritaria/Normal): Normal
CPU requerida: 16
RAM requerida: 32
Servicio critico? (1=SI 0=NO): 1

Solicitud 2 agregada correctamente.
```

Ilustración 10 - Insetado de solicitud crítica

Se verifica la consistencia del modelo de decisiones discretas. A pesar de que el servidor cuenta con CPU y RAM físicos disponibles, el estado de carga ‘ALTA’ restringe la aprobación inmediata de solicitudes ordinarias, forzando el sistema a clasificar el proceso en el conjunto de Espera.

```
--- PROPOSICIONES ---  
P (Servidor operativo): V  
Q (Servicio critico): F  
R (Recursos disponibles): V  
Solicitud 1 -> Espera  
  
--- PROPOSICIONES ---  
P (Servidor operativo): V  
Q (Servicio critico): V  
R (Recursos disponibles): V  
Solicitud 2 -> Aprobada
```

Ilustración 11 - Estado de solicitudes

Como podemos ver cada una de las solicitudes, fue procesada de manera diferente, todo depende de los recursos que requiera cada una de las solicitudes, y si alguna de ellas no cumple con alguna de las proposiciones pasa a ser de Espera.

7. Conclusiones

- Se demostró que la combinación de estructuras de datos lineales y modelos discretos permite simular con precisión entornos informáticos reales, transformando la toma decisiones algorítmicas en métricas cuantitativas de probabilidad y orden parcial.
- La implementación del TDA Cola mediante punteros nativos en C++ garantizó un rendimiento óptimo con una complejidad temporal lineal $O(N)$ y espacial $O(N)$, asegurando que el vaciado y procesamiento de las solicitudes sea predecible y escalable.
- El uso de conjuntos y la teoría de conteo permitieron delimitar formalmente las 12 combinaciones lógicas del espacio muestral, facilitando la clasificación analítica de los criterios de aceptación y espera que gobiernen la infraestructura del servidor.



8. Recomendaciones

- Se recomienda realizar un seguimiento estricto de los operadores new y delete en la gestión de los nodos de la cola, evitando la existencia de punteros colgados o fugas de memoria (memory leaks) que degraden el rendimiento del sistema operativo.
- Se aconseja extender las reglas lógicas del motor de decisiones en futuras iteraciones, incluyendo variables adicionales de hardware (como ancho de banda o almacenamiento) para expandir el universo combinatorio y la profundidad del Diagrama de Hasse.
- Se sugiere que el equipo mantenga la secuencia cronológica de trabajo establecida en la planificación, permitiendo que los datos de salida de la consola sirvan directamente para la validación de los modelos matemáticos en el informe final.

9. Bibliografía

- Castañón, A. G., Rodríguez, J. R., & Del Rosario Gámez Aguilar, M. (2026). Principios de conteo y combinatoria. Ediciones Comunicación Científica. Obtenido de <https://doi.org/10.52501/cc.440.09>
- Dale, N., & Walker, H. M. (1996). Abstract Data Types: Specifications, Implementations, and Applications. D. C. Heath and Company.
- Joyanes Aguilar, L. (2020). Estructuras de datos en C++ (5a ed.). McGraw-Hill.
- Joyanes Aguilar, L., & Zahonero Martinez, I. (2008). Estructuras de datos en C++. McGraw-Hill Interamericana.
- Rosen, K. H. (2012). Discrete Mathematics and its Applications (7th ed.). McGraw-Hill.
- Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.



Williams, N. J. (2024). A survey of congruences and quotients of
partially ordered sets. EMS Surveys in Mathematical Sciences.
doi:<https://doi.org/10.4171/emss/79>